ALGORITHMS PROJECT REPORT

Counting the Tallest Candles

Course	Algorithms
Language	C
Date	14/5/2026

1. Problem Description

Problem: Given an array of integers representing candle heights, determine how many candles are the tallest

1.1 Example

```
Input  :  [4, 4, 1, 3]
Output :  2

Explanation: The maximum height is 4.
             Two candles have height 4, so the answer is 2.
```

1.2 Edge Cases Considered

- Empty array → return 0 (no candles exist)
- Single element → return 1 (it is always the tallest)
- All elements equal → return n (every candle is the tallest)
- One unique maximum → return 1
- Negative heights → algorithm still works correctly

1.3 Constraints

- Array length $n \geq 0$
- Heights are integers (can be negative or zero)
- Goal: find an efficient O(n) solution

2. Algorithm 1 — Iterative Two-Pass

2.1 Idea & Intuition

The simplest approach is to make two separate linear scans of the array:

1. First pass: walk through every element and keep track of the largest value seen so far. After this pass we know the maximum height.
2. Second pass: walk through the array again and count every element that equals that maximum height.

2.2 Pseudo-Code

```
FUNCTION countTallest(candles):
    IF candles is empty THEN RETURN 0

    maxHeight ← -10^8
    FOR EACH h IN candles DO
        IF h > maxHeight THEN maxHeight ← h

    count ← 0
    FOR EACH h IN candles DO
        IF h = maxHeight THEN count ← count + 1

    RETURN count
END FUNCTION
```

2.3 Step-by-Step Trace (input: [4, 4, 1, 3])

Step	Action	Variable	Value
1	Pass 1, index 0	maxHeight	max(-10^8, 4) = 4
2	Pass 1, index 1	maxHeight	max(4, 4) = 4
3	Pass 1, index 2	maxHeight	max(4, 1) = 4
4	Pass 1, index 3	maxHeight	max(4, 3) = 4
5	Pass 2, index 0	count	4 == 4 → count = 1
6	Pass 2, index 1	count	4 == 4 → count = 2
7	Pass 2, index 2	count	1 == 4? No
8	Pass 2, index 3	count	3 == 4? No
9	Return	Result	2

3. Algorithm 2 — Recursive

3.1 Idea & Intuition

The recursive approach mirrors the iterative one, but instead of loops we use function calls

`findMaxRecursive` picks the larger of the current element and the result of recursing on the rest of the array

`countMatchRecursive` adds 1 when the current element equals the target and then recurses

3.2 Pseudo-Code

```
FUNCTION findMax(candles, index):
    IF index = last THEN RETURN candles[index]
    RETURN MAX(candles[index], findMax(candles, index+1))
END FUNCTION

FUNCTION countMatch(candles, index, target):
    IF index = size THEN RETURN 0
    match ← (candles[index] = target) ? 1 : 0
    RETURN match + countMatch(candles, index+1, target)
END FUNCTION

FUNCTION countTallest(candles):
    IF candles is empty THEN RETURN 0
    maxH ← findMax(candles, 0)
    RETURN countMatch(candles, 0, maxH)
END FUNCTION
```

4. Implementation

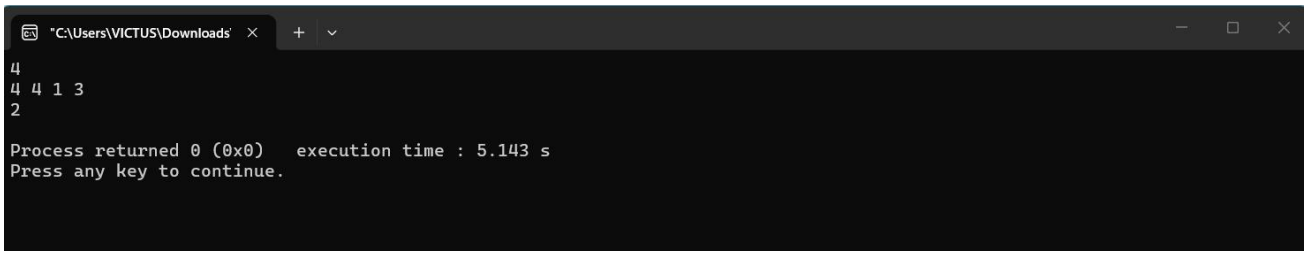
```
#include <stdio.h>

int birthdayCakeCandles(int arr[], int n) {

    int max = arr[0];
    int count = 0;

    for (int i = 0; i < n; i++) {
        if (arr[i] > max) {
            max = arr[i];
            count = 1;
        } else if (arr[i] == max) {
            count++;
        }
    }

    return count;
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];
    for (int i = 0; i < n; i++) {
```

```
        scanf("%d", &arr[i]);
    }

    printf("%d\n", birthdayCakeCandles(arr, n));

    return 0;
```

4.2 : Recursive_ Implementation

```
#include <stdio.h>

typedef struct {
    int max;
    int count;
} Result;

Result findMaxCount(int arr[], int n) {
   if(n==0){
   Result r = {0,0};
   return r;
    }

   else if (n == 1) {
        Result r = {arr[0], 1};
        return r;
    }

    Result prev = findMaxCount(arr, n - 1);

    if (arr[n - 1] > prev.max) {
        Result r = {arr[n - 1], 1};
        return r;
    } else if (arr[n - 1] == prev.max) {
        prev.count++;
        return prev;
    } else {
        return prev;
    }
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    Result res = findMaxCount(arr, n);
    printf("%d\n", res.count);

    return 0;
}
```

4.3. output:



5. Complexity Analysis

Algorithm	Time Complexity	Space Complexity
Iterative Two-Pass	O(n)	O(1)
Recursive	O(n)	O(n)

5.1 explanation:

Iterative Approach

- **Time Complexity: O(n)**
  The array is traversed once (or twice), which results in linear time
- **Space Complexity: O(1)**
  Only a constant number of variables are used (maxHeight, count)

Recursive Approach

- **Time Complexity: O(n)**
  All elements are processed through recursive calls, resulting in linear time
- **Space Complexity: O(n)**
  The recursion depth can reach up to *n*, so the call stack uses linear space

6. Algorithm Comparison

	Iterative	Recursive
Readability	Simple and direct using loops	More compact but needs understanding of recursion
Time complexity	O(n)	O(n)
Space complexity	O(1) — uses fixed memory	O(n) — uses call stack memory
Risk of error	Minimal	May cause stack overflow for large n
Debugging ease	Easy to follow step by step	Harder to trace due to nested calls
Preferred for	Production / large inputs	Academic / learning recursion

7. Edge Cases

Case	Input	Expected
Empty array	N=0	0
Single element	N=1	1
All equal	**{5,5,5,5}**	4(equal n)
Increasing/decreasing	{1,2,3} / {3,2,1}	1
Negative values	{-3,-1,-1,-2}	2(maximum is the least negative number)

End of Report